从Agent到手撕OpenClaw的企业实战

码士集团
www.mashibing.com

讲师：肖斌，发表过超过20000分钟AI大模型课程

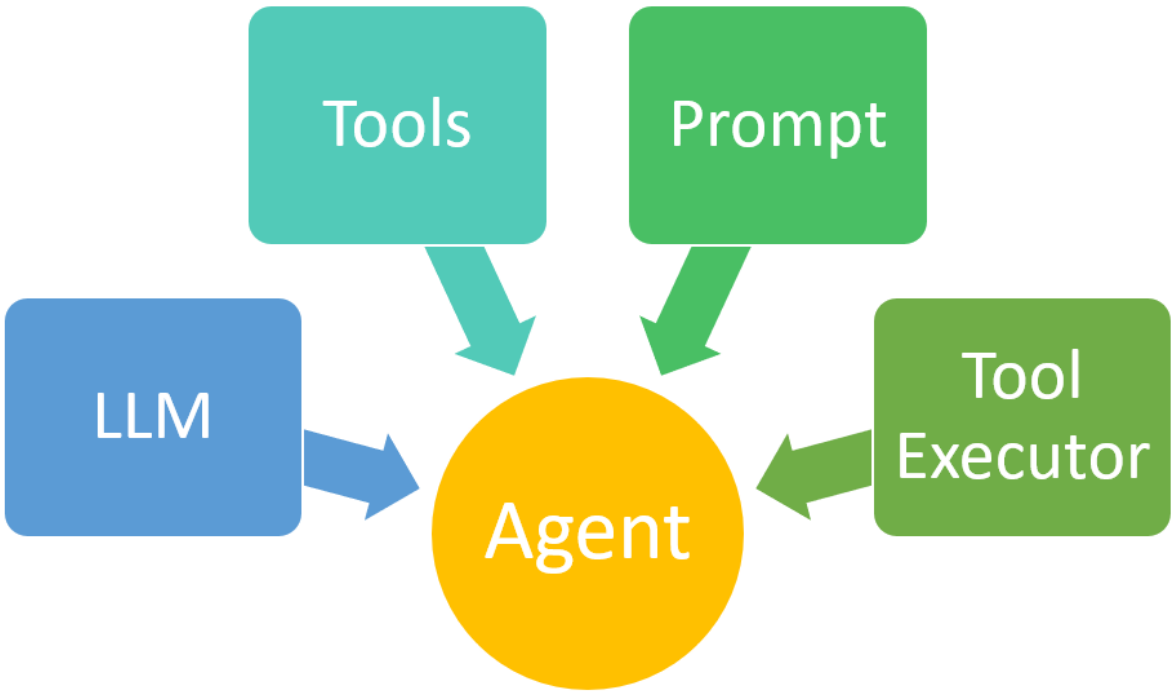
第一章、Agent和OpenClaw的相关概念

1、什么是Agent?

Agent的核心定义是：将大语言模型与工具结合，创建一个能够推理任务、决定使用哪个工具，并迭代式地寻找解决方案，一直到最后完成用户指定任务的系统软件。

可以这样理解：

- **大语言模型** 是系统的“大脑”，负责理解和推理，规划下一步行动。
- **工具** 是系统的“手”和“感官”，能让Agent执行具体的操作（如计算、搜索网络、查询数据库等）。
- **Agent** 是一个协调“大脑”和“手”的自动化工作流。它根据模型对任务的理解，智能地选择并调用一个或多个工具，根据工具的返回结果（观察）再次进行推理，循环往复，直到任务完成或达到停止条件。



一、Agent和传统调用大模型的区别

2025年是“Agent元年”！从“思考”到“行动”的范式跃迁，2026年是Agent爆发年！

特性	AIGC (如 ChatGPT)	AI Agent (如 Manus, OpenC
核心能力	内容生成	任务规划与自主执行
交互模式	被动响应，依赖**提示词**	主动规划，自主决策
输出结果	建议、方案、内容（需人工后续处理）	**可交付的最终成果**（如已发送理好的报表）
与外界交互	有限，主要通过文本	强大，可通过工具调用操作现

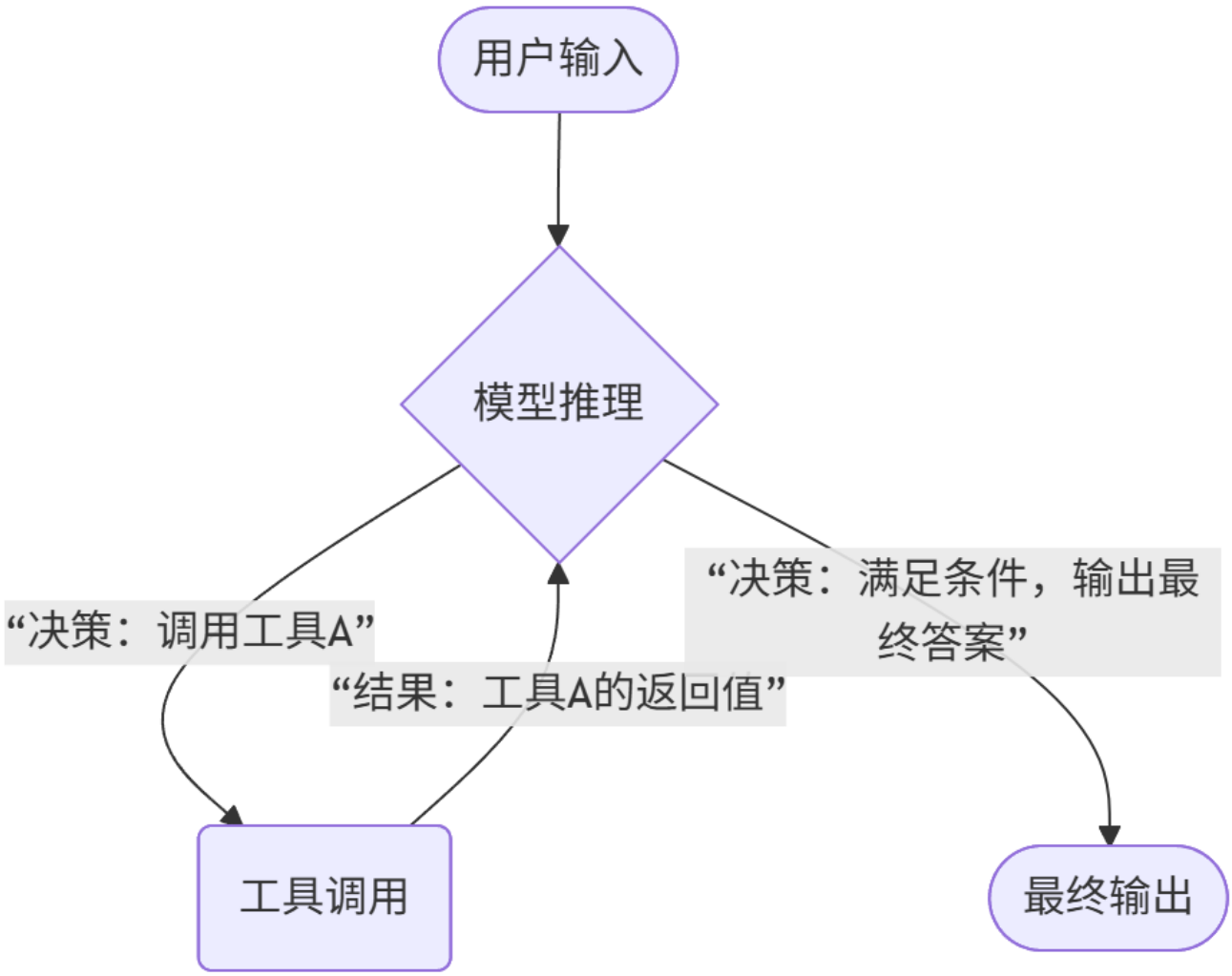
码士集团
www.mashibing.com

二、Agent的运行原理

Agent的工作遵循经典的 **ReAct（Reasoning + Acting， 推理+行动）** 模式，

工作流程说明：

1. **用户输入**：任务开始时，用户向Agent提出问题。
2. **模型推理**：大语言模型分析当前任务和状态，决定下一步做什么。它可能决定“我无法直接回答，需要调用一个工具来获取信息”，并生成一个“工具调用动作”。
3. **工具调用**：Agent执行模型指定的工具，例如进行一次网络搜索或数据库查询，并将执行结果（观察）返回。
4. **迭代**：这个观察结果会和之前的对话历史一起，再次输入给模型。模型基于新的信息进行下一轮推理，可能继续调用其他工具，或认为已经掌握了足够信息，可以生成最终答案。
5. **最终输出**：当模型认为任务已达成或达到迭代限制时，它会生成一个最终的、面向用户的答案。

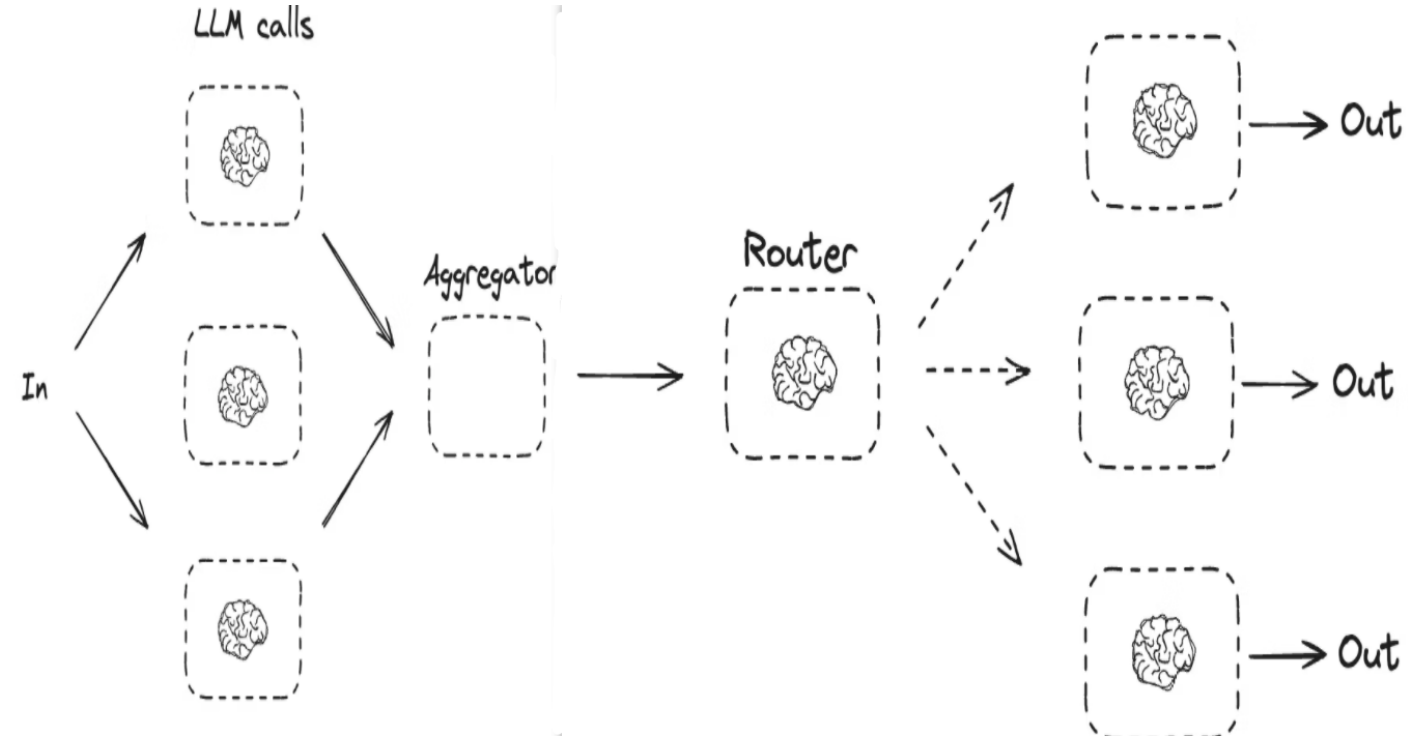


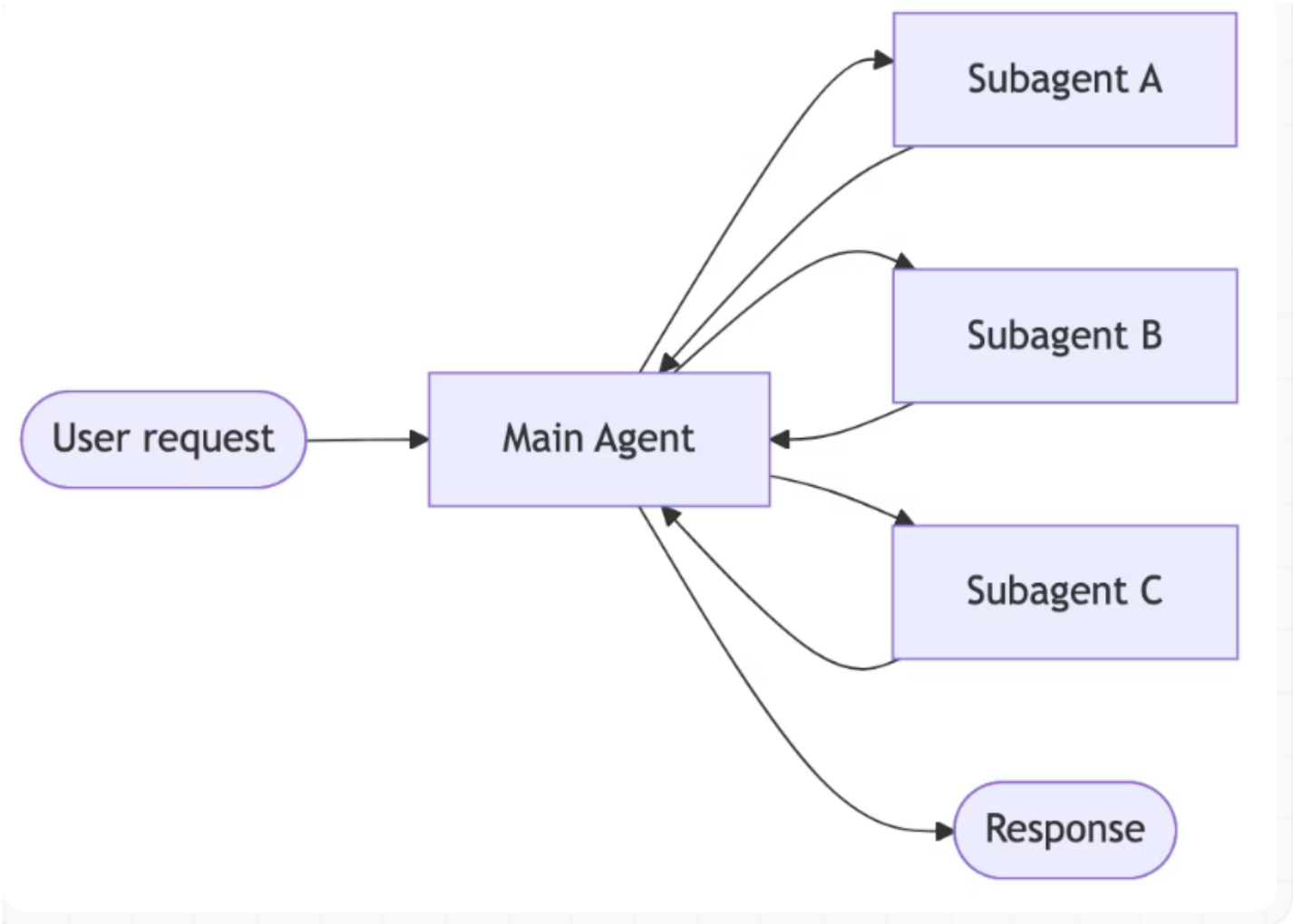
概念	说明	关键点
Agent	模型 + 工具的工作流协调器	遵循ReAct循环，智能调度工具完成任务。
模型	大脑，负责推理	可静态配置，也可动态选择。
工具	手脚，负责执行相关动作	可静态定义，也可基于权限/状态动态过滤或
系统提示	角色设定与行为指南	可静态设置，也可根据上下文动态生成。
状态+上下文	短期记忆	包含消息历史，可扩展以存储自定义信息。
中间件	功能扩展插件	在执行的各个阶段插入自定义逻辑，实现高
结构化输出	控制输出格式	确保Agent的最终回答符合预定义的结构。
Skills	技能包，另外的手脚	可通过渐进式加载来，让Agent动态加载需要 Skill

三、WorkFlow和Agent区别

码士集团
www.mashibing.com

“React的Agent”是一种典型的智能体，而“Workflows”是一个更广泛的概念，代表多种预设的、结构化的LLM编排模式，之前的“React Agent”只是WorkFlow的其中一种简单模式。但Workflows还包括其他**非“React Agent”**的模式，包含多样化的图形结构，例如串联、并行处理、路由等。这些模式也用LLM来做意图决策，但其流程是预设的、结构化的。它是一种更广泛的“编排模式”或“架构模式”的统称。





简单来说：

- 当您需要一个能自己“思考”并决定如何使用工具来完成复杂、开放式任务的助手时，您应该构建一个 **Agent**。
- 当您需要将一系列明确的、结构化的LLM调用步骤自动化时（例如先翻译、再总结、最后润色），您应该设计一个 **Workflow**。Agent可以作为这个Workflow中的一个组件，但并非必须。

2、AI工具/框架分类

搞清楚这些概念！

其核心定位和类别可归纳如下表所示：

码士集团
www.mashibing.com

类别	概念	核心定位与描述	目标用户
AI大模型工具平台 (低代码/可视化应用构建)	**Dify**	一个开源的LLM应用开发平台，提供可视化的方式搭建工作流、RAG引擎，旨在让开发者**快速搭建**Agent或者RAG和部署AI应用。	应用开发者、产品经理、企业团队
	Coze	字节跳动推出的AI Bot开发平台，同样强调低代码和插件生态，用户可通过组装插件、工作流**快速搭建**并发布AI智能体。	个人创作者、产品运营、开发者
	n8n	一个强大的开源工作流自动化工具。有强大的节点生态（含AI节点）和逻辑编排能力，常被用于**搭建**复杂的、多步骤的自动化AI Agent。	业务人员、开发者、自动化工程师
通用的Agent软件 (可执行的智能体应用)	**OpenClaw**	一个开源的多模态AI智能体软件应用，旨在理解和操作图形用户界面（GUI），可模拟人类在电脑上完成各种任务。	研究者、对桌面自动化有需求的业务人员
	Claude Code	它是一个可以被调用的、具有代码执行能力的AI“功能”，可被视为一个原子性的、功能强大的Agent软件。	所有Claude用户
Agent的开发框架 (用于编程构建Agent)	**LangChain**	**最流行的AI应用开发框架之一**。提供模块化组件（Models, Tools, Memory等）和“链”的抽象，用于编排基于LLM的应用程序和基础Agent。	AI应用开发者
	LangGraph	构建在LangChain之上的框架，用于创建**有状态的、多智能体**工作流。其核心是通过“图”来定义Agent或工具之间的循环和条件流转，适合构建复杂、长期运行的Agent系统。	需要构建复杂协作Agent的开发者
	DeepAgents	LangChain生态中一个**专注于构建“深度”或“长周期”任务Agent**的框架。它提供了虚拟文件系统、任务规划、子Agent委托、高效的上下文管理等高级能力，专门为处理需要长时间运行、多步骤交互的复杂任务而设计。	需要构建处理复杂、长期任务Agent的开发者
	Spring AI	Spring生态系统的AI集成框架。它为Java/Kotlin开发者提供了一套统一的API，来调用各种大模型并提供RAG、函数调用等基础组件，方便在Spring应用中集成AI能力。	Java/Spring生态的开发者

平台 vs. 框架 vs. 软件

- **平台 (Platform)**：如 **Dify, Coze**。提供**开箱即用**的环境，通常有Web界面，通过可视化拖拽和配置来搭建应用Agent。它们负责底层的基础设施（如模型调度、部署），用户更关注业务逻辑组装。**核心是“搭建”**。

- **框架 (Framework)**：如 **LangChain, LangGraph, Spring AI, DeepAgents**。是一套**代码库和规范**，为开发者提供构建模块和最佳实践。开发者需要编写代码来使用它们，拥有更高的灵活性和控制力。**核心是“开发”**。
- **软件/应用 (Software)**：如 **OpenCode, OpenClaw**。是一个**可独立运行的程序**，拥有自己的运行时和用户界面，用于完成特定类型的任务（自动化、GUI操作）。用户通过配置和使用它来直接产生价值。**核心是：用**

码士集团
www.mashibing.com

3、OpenClaw是什么?

OpenClaw（曾用名 Clawdbot，中文昵称“小龙虾”）是一款 **运行在本地设备上的开源通用型的AI智能体（Agent）**。它由奥地利独立开发者 Peter Steinberger（PSPDFKit创始人）于2025年底创建，在2026年初迅速爆红，GitHub星标数在短时间内突破27万，成为全球最受关注的开源项目之一。

模块化与Skills系统架构：

- OpenClaw的能力通过 **Skills（技能）** 进行扩展。每个Skill都是一个包含自然语言指令的文件夹，相当于一份“标准作业程序（SOP）”，教会AI处理特定任务。
- 采用 **“渐进式信息披露”** 策略：启动时只加载Skill的元数据描述，当任务匹配时，才动态加载详细指令，极大优化了Token使用效率。
- 拥有庞大的社区技能市场 **ClawHub**，提供超过500个现成技能，覆盖办公、开发、数据分析等场景。

```
skills/
├── langgraph-docs
│   └── SKILL.md
└── arxiv_search
    ├── SKILL.md
    └── arxiv_search.py # code for searching arXiv
```

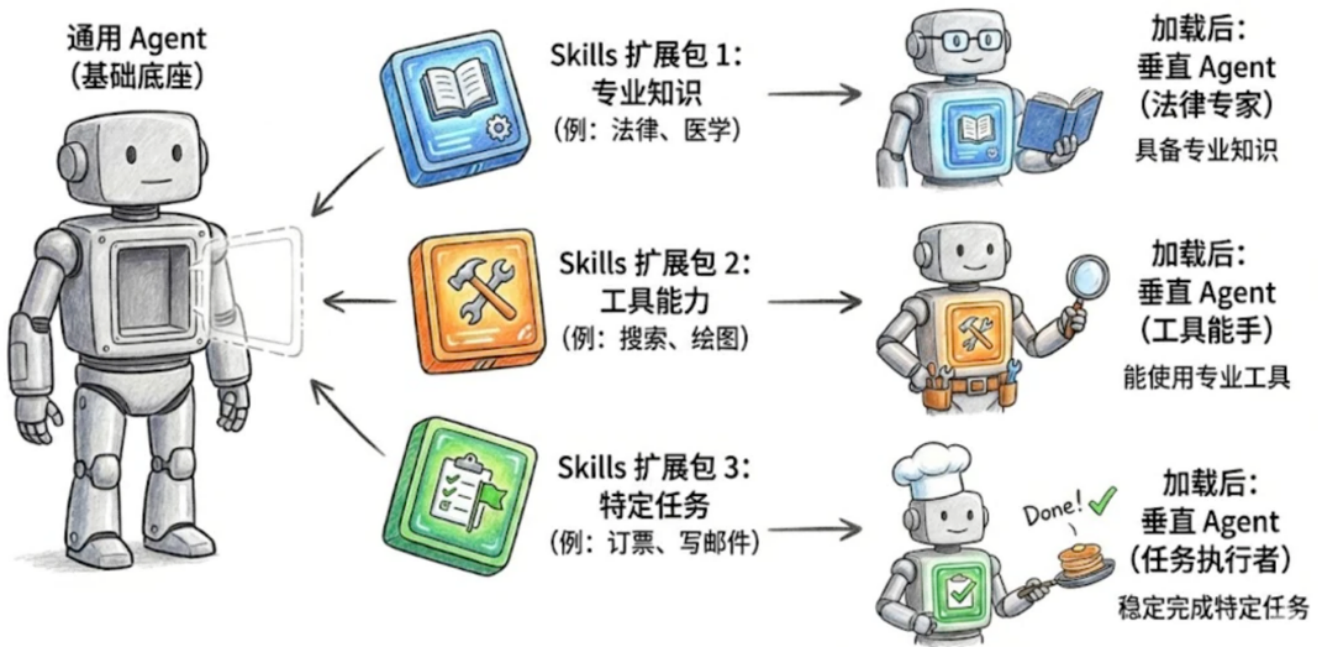
这是两个Skill

争议与风险：

- **极高的安全风险**：这是最受关注的问题。它需要**极高的系统权限**，一旦被恶意利用或遭受“提示注入攻击”，可能导致敏感数据泄露、系统被完全控制。国家网络与信息安全信息通报中心已发布专项风险预警。
- **供应链投毒**：社区技能市场（ClawHub）中，**约10.8%的Skill技能包被检测出包含恶意代码**，用户安装技能时极易“中招”。
- **行为不可控**：AI在执行复杂指令时可能发生“权限失控”，无视用户后续指令，进行危险操作（如删除文件）。
- **使用成本与门槛**：复杂任务消耗的Token量巨大，账单可能失控；同时其部署和维护仍有技术门槛，对普通用户并不友好。

码士集团
www.mashibing.com

4、OpenClaw的影响：



OpenClaw的崛起标志着AI从 “对话与内容生成” 时代，正式迈入 “自主执行与行动” 的Agent时代。它不再是一个玩具，而是一个切实的 生产力工具原型。它的模块化与Skills系统架构证明：***这是一***

套新的Agent软件开发中可行的方案！

“Agent + Skills” 架构是一种高度模块化、可扩展的设计范式，用于构建复杂且专业的AI智能体系统。其核心思想是将智能体的基础推理与执行能力（Agent）与其专业技能和工作流（Skills）解耦。

这种架构的具体构成及其为开发AI Agent系统软件带来的显著优势。该架构通常包含两个核心层级：

1. **智能体核心层**
   - 这是系统的“通用大脑”，由框架（如LangChain，DeepAgents）提供。它负责基础的任务理解、规划、决策、工具调用和上下文管理。
   - 一个强大的智能体核心应具备**规划能力**、**环境交互能力**（虚拟文件系统工具 `ls`，`read_file`， `write_file` 等）、**任务分解能力**（子Agent）以及关键的**长上下文管理能力**（通过内容卸载与摘要化来维持长期记忆）。这是支撑复杂任务的基础。
2. **技能模块层**
   - 这是系统的“专业化技能包”。每个Skill是一个独立的模块，包含完成特定领域任务所需的所有知识、指令和资源。
   - 一个技能是一个目录，其核心是一个 `SKILL.md` 文件。该文件包含元数据（名称、描述）和详细的、循序渐进的指令，并可引用其他脚本、模板等资源。

采用“Agent + Skills”的模块化架构，能为系统设计和开发带来多方面的重要优势：

1. 极致的可扩展性与灵活性：

   新能力的添加不再需要修改智能体核心代码。开发者或领域专家可以像“开发插件”一样，独立地创建和封装新的Skill（只需遵循 `SKILL.md` 规范）。系统能力可以通过累积Skills的方式无限扩展。

2. 卓越的Token与上下文管理效率：

   这是“渐进式披露”机制带来的最直接好处。智能体启动时无需将大量、可能无关的专业指令全部载入系统提示词，极大节省了初始上下文窗口。只有在确需时，才支付读取特定技能详细内容的Token成本。这直接支撑了更复杂、集成更多专业知识的智能体成为可能。

3. 清晰的关注点分离与专业分工：

   智能体核心开发者专注于提升基础能力：优化规划算法、增强工具使用稳定性、改进上下文管理策略等（如优化内容卸载策略）。**技能开发者**则专注于领域知识：将最佳实践编写成精准、可重复执行的指令，无需关心底层智能体如何调度与记忆。

4. 提升系统的可维护性与可靠性：

   技能模块化使得调试和更新变得更容易。如果某个领域任务执行出错，可以独立检查和修复对应的Skill指令，而不会影响智能体核心或其他技能。技能的版本管理也变得可行。

5. 实现动态的能力组合：

对于复杂任务，智能体核心可以规划并串联调用多个Skills来协同完成。这种动态组合能力使得智能体能够应对远超单个技能范围的复杂、跨领域工作流。

但是：“Agent + Skills”架构，并不是说：Agent中没有Tool（工具）！

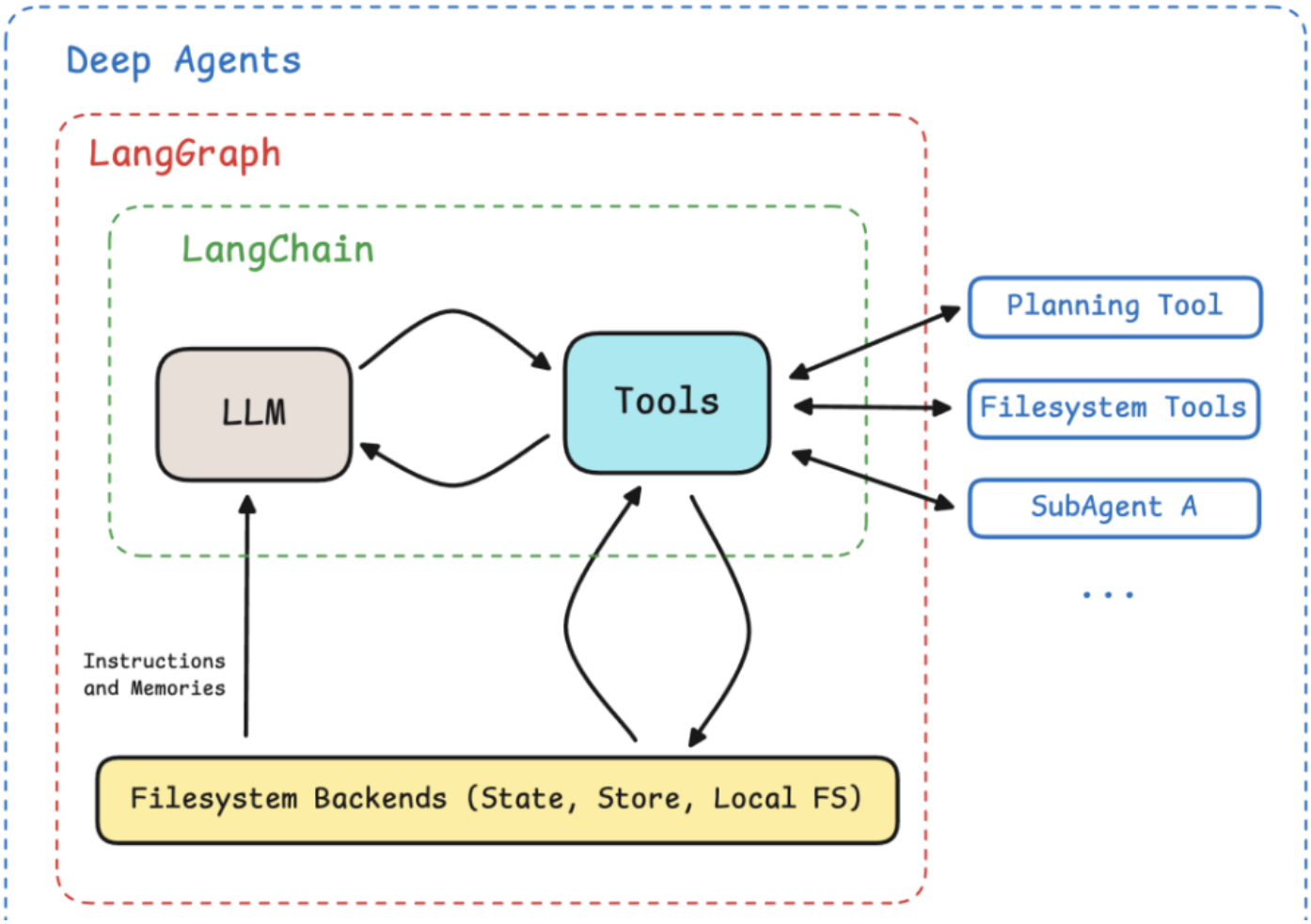
码士集团
www.mashibing.com

第二章、DeepAgents框架初探

1、DeepAgents框架的核心

DeepAgents 是 **LangChain 团队** 在 LangChain 和 LangGraph 之上推出的 **第三个独立开源项目**，是一个 **“开箱即用”的深度智能体框架。**

一、DeepAgent框架的由来和定义



Deep Agents 并非颠覆性技术创新，而是将业界验证过的最佳实践打包成开箱即用的框架。它的灵感来源于 Claude Code、Deep Research、Manus 等成功应用，这些应用证明了四个关键点：

· **Agent 需要规划能力** —— 不能上来就开干，得先拆解任务

· **Agent 需要文件系统** —— 长对话中，得有地方放中间结果，需要加载本地环境

· **Agent 需要子 Agent** —— 单体 Agent 会被上下文撑爆

· **Agent 需要子 Skills** —— Agent 功能可以弹性扩展，并且支持模块化开发。

LangChain 团队的想法很简单：既然这些经验都验证过了，为什么不打包成一个库？于是就有了 Deep Agents。

DeepAgents是一个Langchain团队在2025年11月推出的新框架，专门用于构建能够处理**复杂、长周期任务**的智能体（Agent）的**框架**。它的核心目标是让开发者能够更容易地创建出功能强大、可靠且可长期运行的AI智能体。

二、DeepAgents 的核心能力

Deep Agents的核心能力模块：

模块一：规划能力（Planning）

- **功能**：提供一个 `write_todos` 工具，允许智能体维护一个结构化的任务清单。
- **作用**：帮助智能体跟踪多个任务及其状态（`'pending'`，`'in_progress'`，`'completed'`），以组织复杂的多步骤工作，尤其适用于长周期任务。

模块二：虚拟文件系统（Virtual Filesystem）

- **功能**：提供一个可配置的虚拟文件系统，支持 `ls`（列出文件）、`read_file`（读取文件，支持图片和大型文件）、`write_file`（创建文件）、`edit_file`（编辑文件）、`glob`（模式匹配）、`grep`（内容搜索）和 `execute`（执行命令，仅在沙盒后端可用）等工具。
- **作用**：为其他能力（如技能、记忆、代码执行）提供基础存储和操作层，也可用于构建自定义工具。

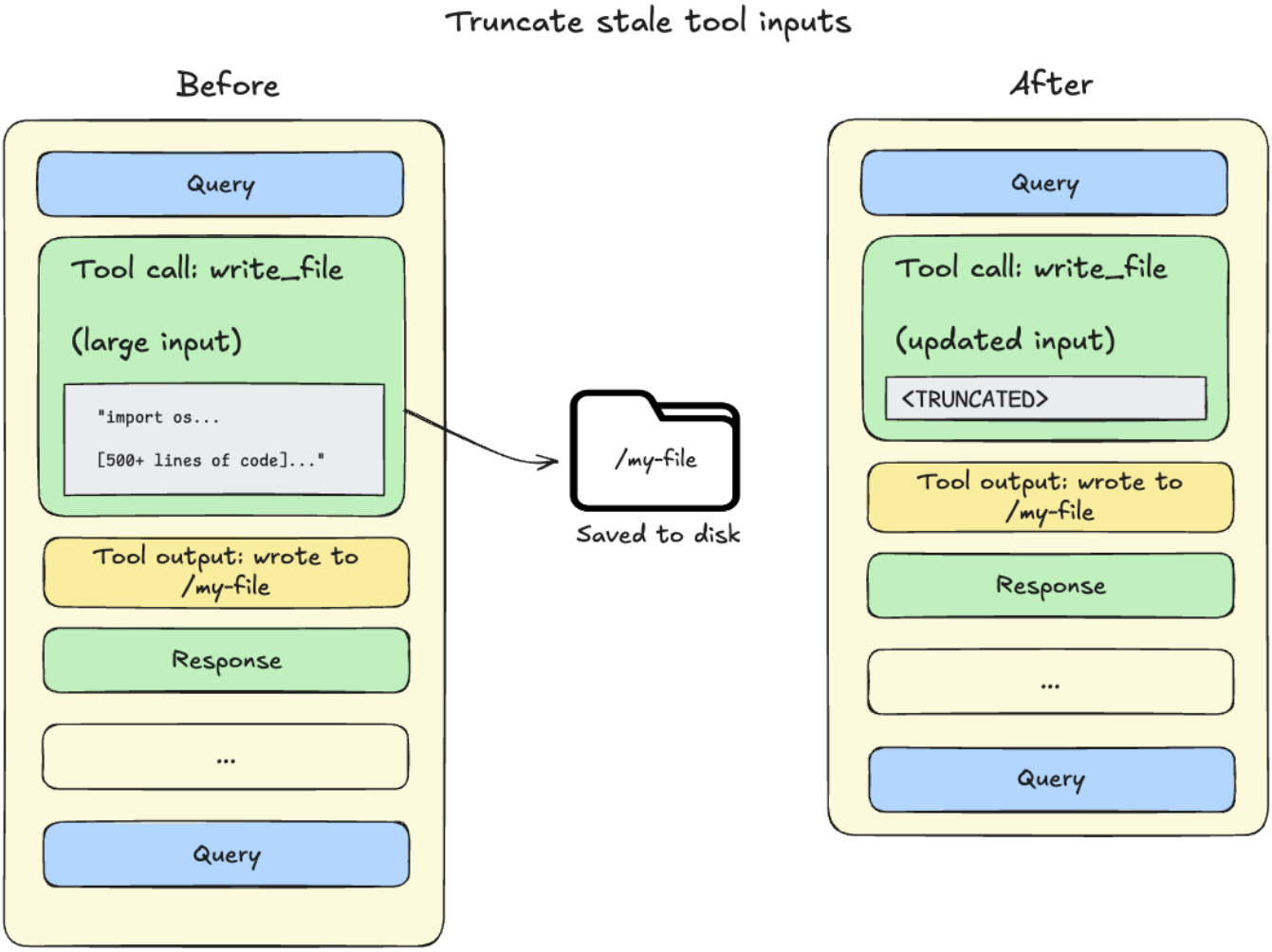
模块三：任务委托/子智能体（Task Delegation / Subagents）

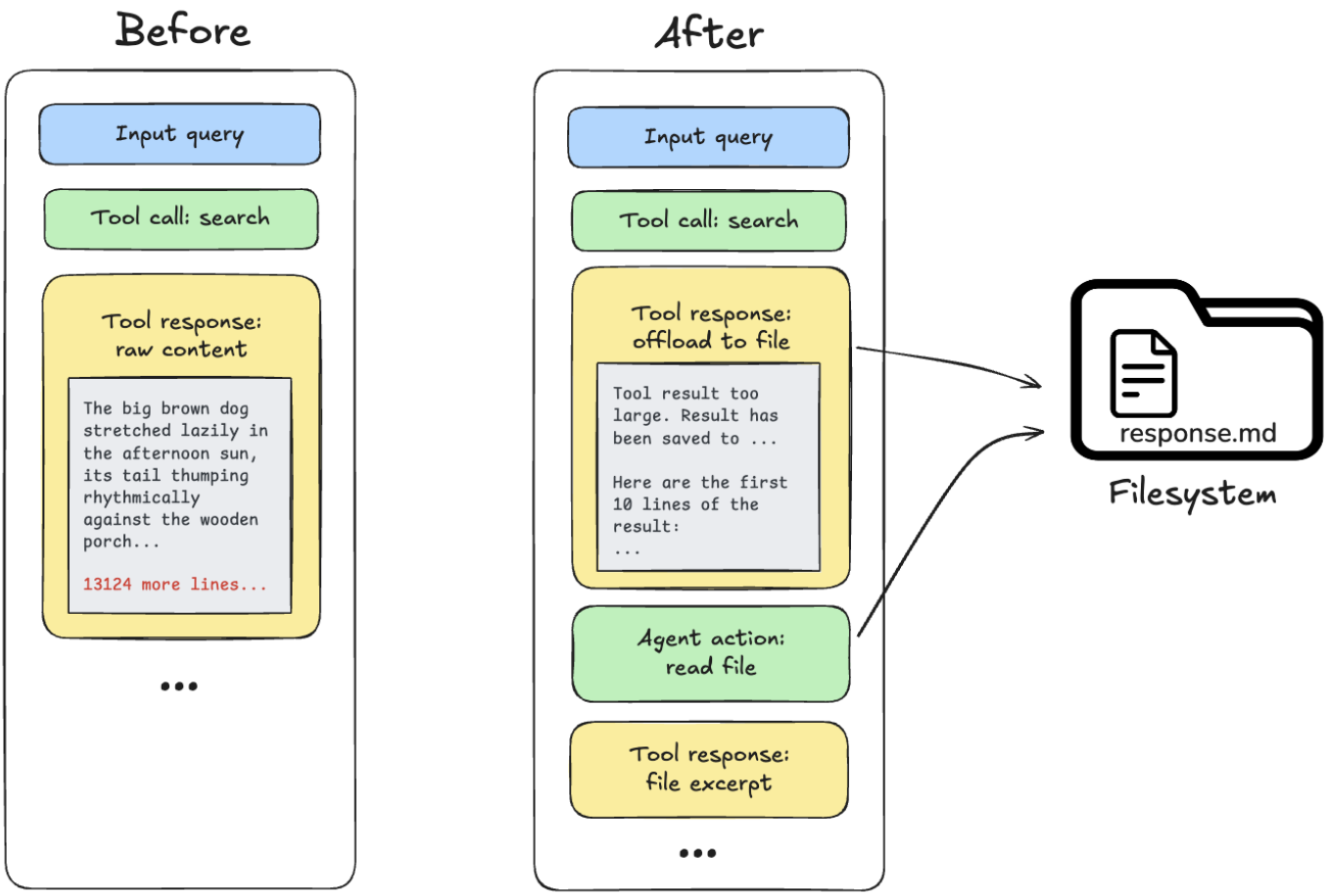
- **功能**：允许主智能体创建临时的“子智能体”来处理独立的多步骤任务。
- **作用**：
  - **上下文隔离**：子智能体的工作不会干扰主智能体的上下文。
  - **并行执行**：多个子智能体可以并发运行。
  - **专业化**：可以为子智能体配置不同的工具集。
  - **Token高效**：将大型子任务的上下文压缩为单一结果报告返回给主智能体。

模块四：上下文管理（Context Management）

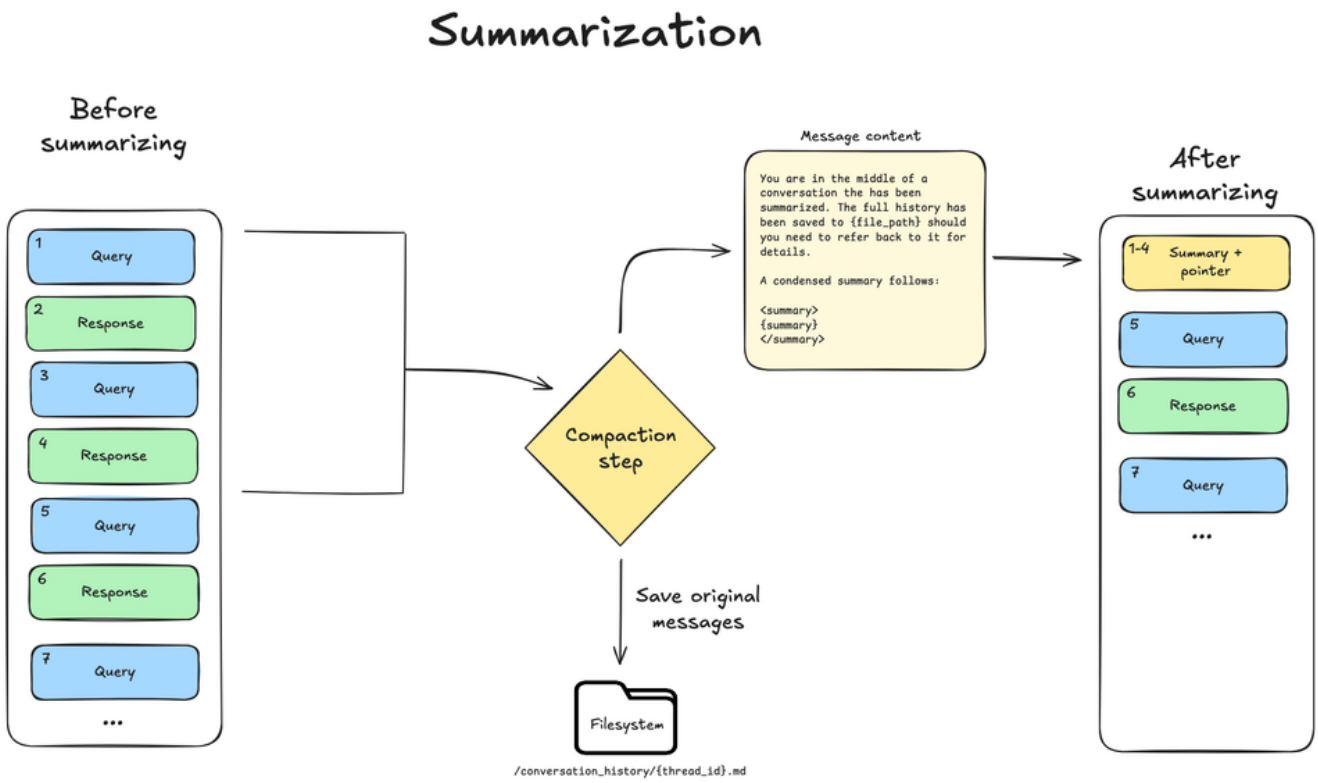
这是智能体处理长对话和复杂任务的关键，通过多种技术管理其工作记忆（上下文），确保不超出模型的处理窗口限制。主要包括：

- **输入上下文**：包括系统提示词、待办列表、记忆、技能、文件系统工具说明等，在智能体启动时注入。
- **运行时上下文压缩**：
  - **卸载大型工具输入和结果**：当工具调用的输入或结果超过一定大小时（默认20，000 tokens），智能体会自动将其内容保存到文件系统，并在对话历史中用文件路径和摘要替换，以节省上下文空间。当会话上下文超过模型窗口的85%时，会触发此操作。





- **摘要化 (Summarization)**：当上下文大小达到极限且无可卸载内容时，智能体会将过去的对话历史总结为一段结构化摘要，替换原有的详细历史，同时将完整历史保存到文件系统。



- **长期记忆 (Long-term memory)**：通过混合存储后端，允许智能体将特定路径（如 `/memories/`）下的信息持久化存储，使其能够跨不同的会话和线程访问，用于存储用户偏好、累积知识等。

模块五：代码执行（Code Execution）

- **功能**：当使用沙盒后端时，会向智能体暴露一个 `execute` 工具，允许其在隔离环境中运行 shell 命令。
- **作用**：使智能体能够安装依赖、运行脚本、执行代码，从而完成更复杂的任务，同时保证了主机系统的安全。

模块六：人工介入（Human-in-the-loop, HITL）

- **功能**：通过 `interrupt_on` 参数配置，可以在指定的工具调用前暂停智能体执行，等待人工批准或修改输入。
- **作用**：为具有破坏性或昂贵的操作（如编辑文件、调用API）增加安全闸门，便于交互式调试和指导。

模块七：技能包加载和管理（Skills）

- **功能**：采用“渐进式披露”，仅在需要时加载，以减少系统提示的 token 占用。适用于任务特定、内容可能很多的场景。
  - - **渐进式披露**：这是技能加载的核心机制。智能体启动时，**仅读取**所有 `SKILL.md` 文件的**元数据部分**。当接收到用户提示后，智能体根据元数据中的描述判断是否有相关技能可用。只有匹配到相关技能时，才会去**读取该技能文件夹的完整内容**（包括 `SKILL.md` 的详细指令和其他资源文件）。这大大减少了初始上下文的 token 消耗。
  - - **智能体使用流程**：匹配（根据描述）→ 读取（完整技能内容）→ 执行（遵循技能指令）。
- **作用**：Skills 是用于扩展 Deep Agent 能力的可重用模块。

三、框架对比

维度	LangGraph	LangChain	Deep Agents
定位	智能核心开发框架	智能体开发的基础框架	复杂智能体开发框架
抽象层级	低层（自己造轮子）	基础层（积木块）	高层（开箱即用）
核心抽象	StateGraph、Node、Edge	Chain、Runnable、Tool	DeepAgent、Middleware、Backend
规划能力	需要自己实现	非原生支持	✅ write_todos内置
文件系统	需要自己实现	非原生支持	✅ 完整文件系统工具集
子代理委托	可通过子图实现	非原生支持	✅ task工具内置
长期记忆	需要自己配置	基础记忆抽象	✅ 原生支持 + 外部持久化
上下文管理	通过图结构控制	依赖模型上下文窗口	✅ 文件系统 + 持久化记忆
学习曲线	陡峭	中等	陡峭
灵活性	极高	高	中等（但很实用）
适用对象	复杂企业流程序列化开发者	库开发者/底层构建	应用开发者/企业级解决方案

DeepAgents的核心定位是：DeepAgents 是 **LangChain 生态中面向“复杂任务”的“上层应用框架”**。

专门为构建能够处理复杂、多步骤、长时间运行任务的“深度”智能体（Deep Agent）而设计。它并非要取代 LangChain 或 LangGraph，而是在它们提供的底层运行时和基础抽象之上，**封装了业界已验证的最佳实践**（如 Claude Code、Manus 等应用的模式），将这些复杂能力标准化、产品化，模块化，工程化。

我的开发建议和Agent生态分层

- **底层框架**（如LangChain）解决通用性问题。提供 `create_agent`，tool, LLM调用等高层抽象和标准组件。
  - **简单、直接的任务** → 使用 `LangChain` 的 `create_agent`。
- **专项框架**（如LangGraph for 多Agent， DeepAgents for 长任务）在通用框架上针对特定场景做深度优化。
  - LangGraph：提供最底层的状态管理、工作流编排。**需要完全自定义、复杂控制流的工作流** → 使用 `LangGraph` 从底层构建。
  - DeepAgents：专为 **“重任务”** 设计，不适合简单聊天机器人，**需要处理复杂、多步骤、长时间运行的“深度”任务** → 选择 `DeepAgents`。

    其典型场景包括：

    - **深度研究与报告撰写**：自动进行多轮搜索、阅读、分析并生成长篇报告。
    - **全栈代码生成与重构**：在沙箱中编写、测试、调试和重构整个代码库。
    - **复杂数据分析流水线**：连接数据库、执行查询、处理中间文件并生成可视化图表。
    - **自动化运维与业务流程**：操作文件、执行命令、编排需要多角色协作的复杂工作流。

2、开发第一个OpenClaw的Agent

步骤：

①**需要一个LLM；**

②**开发一个Agent，并且需要有复杂任务规划能力；**

③**准备一个Skills目录；**

④**准备一个本地沙箱做安全隔离；**

⑤**准备一个适当的系统提示词；**

⑥**准备记录上下文；**

⑦**开发一个Skill技能包；**

⑧**执行Agent，并流式输出；**

⑨**再追加新的Skill技能包；**

⑩…